Software Development Life Cycle (SDLC)

1. Introduction

Software is not created in a single step. Developing a reliable software application requires multiple activities, including understanding requirements, designing the system, writing code, testing functionality, deploying the application, and maintaining it.

To organize these activities systematically, software development teams follow a structured process known as the **Software Development Life Cycle (SDLC)**.

SDLC provides a framework that guides a software project from its initial idea to its deployment and ongoing maintenance.

2. Definition of SDLC

The **Software Development Life Cycle (SDLC)** is a structured process followed by software development teams to plan, design, develop, test, deploy, and maintain software applications in a systematic and organized manner.

It defines the different phases involved in software development, the activities performed during each phase, and the expected deliverables.

The primary objective of SDLC is to develop high-quality software that satisfies user requirements within the planned time, budget, and resources.

3. Why Do We Need SDLC?

Consider developing an online shopping application.

Without a proper development process, developers might start writing code without completely understanding the requirements. Important functionalities could be missed, design decisions might be inconsistent, and defects could be discovered very late.

SDLC helps prevent such problems by providing a defined process.

Its major purposes include:

- Understanding and documenting customer requirements.
- Planning project activities, resources, time, and cost.
- Designing a suitable software architecture.
- Developing software in an organized manner.
- Identifying defects before delivering the application.
- Ensuring that the final product satisfies business requirements.
- Supporting future maintenance and improvements.

4. Phases of SDLC

SDLC is commonly explained through the following six major phases.

4.1 Planning and Requirement Analysis

This is the initial phase in which the project objectives and customer requirements are identified and analyzed.

The development team communicates with stakeholders to understand what the software should accomplish.

Activities:

- Understanding business objectives.

- Gathering requirements from customers and stakeholders.
- Identifying project scope.
- Estimating development time and cost.
- Identifying technical and business risks.
- Evaluating project feasibility.

Deliverables:

- Business Requirement Document (BRD).
- Software Requirement Specification (SRS).
- Project plan.
- Feasibility analysis.

Example:

For an online shopping application, requirements may include user registration, login, product search, shopping cart, payment, and order confirmation.

4.2 System Design

After understanding the requirements, the team determines how the software will be built.

The architecture, components, database structure, user interfaces, and interactions between different modules are designed during this phase.

Activities:

- Selecting the system architecture.
- Designing database tables and relationships.
- Designing user interfaces.
- Defining APIs and system components.
- Identifying technologies and frameworks.

Deliverables:

- High-Level Design (HLD).
- Low-Level Design (LLD).
- Database design.
- Architecture diagrams.
- UI/UX designs.

Example:

For the shopping application, the team may design separate modules for authentication, product management, cart management, payments, and orders.

4.3 Implementation (Coding)

During this phase, developers convert the approved designs into executable software by writing code.

Different developers may work on different modules according to the project architecture and development plan.

Activities:

- Writing source code.
- Developing application modules.
- Implementing business logic.
- Integrating databases and APIs.
- Performing code reviews.
- Following coding standards.

Deliverables:

- Source code.
- Developed application modules.
- Unit-level implementation.
- Build artifacts.

Example:

Developers implement Java Spring Boot APIs for registration, login, product retrieval, cart operations, and order management.

4.4 Testing

After implementation, the software is tested to verify whether it behaves as expected and satisfies the specified requirements.

The testing team identifies defects, reports them to developers, and verifies fixes.

Activities:

- Preparing test scenarios and test cases.
- Executing test cases.
- Performing functional testing.
- Performing integration testing.
- Performing system testing.
- Reporting and tracking defects.
- Retesting fixes and performing regression testing.

Deliverables:

- Test Plan.
- Test Cases.
- Test Execution Results.
- Defect Reports.
- Test Summary Report.

Example:

Testers verify whether users can register successfully, whether invalid passwords are rejected, whether products can be added to the cart, and whether orders are generated correctly.

4.5 Deployment

Once the software has been tested and approved for release, it is deployed to the target environment so that intended users can access it.

Activities:

- Preparing the deployment environment.
- Configuring servers and databases.
- Deploying application builds.
- Performing deployment verification.
- Monitoring the application after release.

Deliverables:

- Deployed application.
- Release notes.
- Deployment documentation.
- Environment configuration.

Example:

The shopping application is deployed to a production server and made available to customers through its website.

4.6 Maintenance

Software development does not end after deployment. Applications require continuous monitoring, corrections, improvements, and updates.

Maintenance ensures that the application continues to operate reliably as business needs and technologies change.

Activities:

- Fixing production defects.
- Improving application performance.
- Updating security mechanisms.
- Adding new functionalities.
- Updating dependencies.
- Monitoring application stability.

Deliverables:

- Bug fixes.
- Software updates.
- Improved application versions.
- Maintenance documentation.

Example:

After releasing the shopping application, the company may introduce new payment methods, improve product search performance, and fix issues reported by customers.

5. SDLC Workflow

The following diagram illustrates a commonly used sequential representation of SDLC.

Planning and Requirement Analysis

|

v

System Design

|

v

Implementation

(Coding)

|

v

Testing

|

v

Deployment

|

v

Maintenance

The phases are connected because the output of one phase provides the foundation for the next phase. However, the exact sequence and feedback between phases depend on the SDLC model adopted by the organization.

6. Roles Involved in SDLC

Different professionals contribute to the software development process.

Role	**Responsibility**
Business Analyst	Understands business needs and gathers requirements.
Project Manager	Plans and coordinates project activities.
Software Architect	Defines the overall system architecture.
UI/UX Designer	Designs the user interface and user experience.
Software Developer	Designs and implements software functionality.
Software Tester / QA Engineer	Verifies software quality and identifies defects.
DevOps Engineer	Supports build, deployment, infrastructure, and delivery processes.

7. Advantages of SDLC

- Provides a structured approach to software development.

- Improves project planning and coordination.
- Helps identify risks at an early stage.
- Improves communication among team members.
- Supports better documentation.
- Helps maintain software quality.
- Makes project progress easier to monitor.

8. SDLC and Its Relationship with STLC

SDLC covers the complete software development process, whereas STLC focuses specifically on software testing activities.

STLC is closely associated with the testing phase of SDLC, although testing activities can begin much earlier during requirements and design reviews.

For example, while developers design a login feature, testers can already review the requirements and identify possible test scenarios.

Both SDLC and STLC work together to deliver reliable software.

9. Common SDLC Models

Organizations use different models to organize and manage the SDLC process.

Some commonly used models include:

1. Waterfall Model.
2. Agile Model.
3. V-Model.
4. Spiral Model.
5. Iterative Model.
6. Incremental Model.

Each model follows a different approach to organizing development activities, managing requirements, handling changes, and delivering software.

Waterfall and Agile are discussed in separate documents.

10. Conclusion

The Software Development Life Cycle provides a systematic framework for developing and maintaining software applications.

By organizing software development into defined phases, SDLC helps teams understand requirements, plan work, design solutions, implement functionality, verify quality, deploy releases, and maintain applications.

Understanding SDLC is essential for software developers, testers, SDETs, project managers, and anyone involved in the software development process.